

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



COURSE: SOFTWARE ARCHITECTURE
TOPIC: Online movie streaming platform (Nozie platform)

Instructor: **Vũ Quang Dũng**

Group 03: **Đặng Nhất Nhất** (23010345)

Class Section: CSE703110-1-2-25(N02)

Ha Noi, Feb 2026

Table of Contents

1. Testing & Evidence.....	4
1.2 Non-Functional Testing.....	6
1.3 End-to-End Testing (Flutter ⇔ Gateway ⇔ Microservices).....	6
2. Architectural Assessment & Lessons Learned	7
2.1 Fulfillment of FRs / NFRs / ASRs	7
2.2 Monolith vs. Microservices: Trade-off Analysis	7
2.3 Advantages, Limitations, and Remaining Risks.....	8
2.4 Lessons Learned	8
3. Conclusion & Future Direction	8
3.1 General Conclusion for Nozie Project	8
3.2 Functional and Architectural Extension Roadmap	9

ASSIGNED ROLE: Quality Assurance (QA) & Documentation

Main Tasks: Responsible for quality assurance, system testing, architectural validation, and documentation of testing results.

1. Testing & Evidence

1.1 Core Functional Test Scenarios

Systematic testing was performed across all microservices to ensure business logic integrity. The following table summarizes the key functional test cases executed via Postman and Automated Unit Tests

Table 1.1 – Key Functional Test Results

ID	Feature	Test Scenario	Expected Result	Status
FT-01	Authentication	Login with valid credentials	Receive JWT Access & Refresh tokens	Passed
FT-02	Catalog	Fetch movie list through Gateway	200 OK with JSON array of movies	Passed
FT-03	Streaming	Access Premium movie with Free account	403 Forbidden / Payment Required prompt	Passed
FT-04	Payment	Complete Stripe payment flow	Payment Service triggers "SubscriptionActivated" event	Passed

ID	Feature	Test Scenario	Expected Result	Status
FT-05	Notification	Event-driven Email delivery	Notification Service consumes RabbitMQ message	Passed

Filter by: All 29 Passed 29 Failed 0 Skipped 0

- ✔ Movie Service/1. Catalog - List & Filter/1.1 Get Movies (with filters) 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.2 Get Latest Movies 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.3 Search Movies 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.4 Get Movies by Type 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.5 Get Movies by Genre 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.6 Get Movies by Country 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.7 Get Movies by Year 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.8 Get Trending Movies 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.9 Get Free Movies 200 - OK
- ✔ Movie Service/1. Catalog - List & Filter/1.1a Get Movies (full filter) 200 - OK
- ✔ Movie Service/2. Catalog - Detail/2.1 Get Movie by ID 200 - OK
- ✔ Movie Service/2. Catalog - Detail/2.2 Get Movie by Slug 200 - OK
- ✔ Movie Service/3. Catalog - Metadata/3.1 Get Genres 200 - OK
- ✔ Movie Service/3. Catalog - Metadata/3.2 Get Countries 200 - OK
- ✔ Movie Service/3. Catalog - Metadata/3.3 Get Years 200 - OK
- ✔ Movie Service/4. Catalog - CRUD/4.1 Create Movie 200 - OK
- ✔ Movie Service/4. Catalog - CRUD/4.2 Update Movie 200 - OK
- ✔ Movie Service/4. Catalog - CRUD/4.3 Delete Movie 200 - OK
- ✔ Movie Service/5. Streaming/5.1 Get Play URL (by ID) 200 - OK
- ✔ Movie Service/5. Streaming/5.2 Get Play URL (by slug) 200 - OK
- ✔ Movie Service/5. Streaming/5.3 Get Episodes (by ID) 200 - OK
- ✔ Movie Service/5. Streaming/5.4 Get Episodes (by slug) 200 - OK
- ✔ Movie Service/5. Streaming/5.5 Increment View (by ID) 200 - OK
- ✔ Movie Service/5. Streaming/5.6 Increment View (by slug) 200 - OK
- ✔ Movie Service/6. Reviews (UC13)/6.1 Rate Movie 200 - OK
- ✔ Movie Service/6. Reviews (UC13)/6.2 Get Movie Reviews 200 - OK
- ✔ Movie Service/7. Recommendations (UC26/27)/7.1 Get Personalized Recommendations 200 - OK
- ✔ Movie Service/7. Recommendations (UC26/27)/7.2 Get Similar Movies 200 - OK
- ✔ Movie Service/7. Recommendations (UC26/27)/7.3 Get Trending Recommendations 200 - OK

1.2 Non-Functional Testing

Architectural quality is validated through performance, resilience, and scalability testing.

1.2.1 Latency Analysis (API Gateway Overhead)

We measured the latency of requests through the API Gateway versus direct service calls to ensure routing overhead remains under the 50ms threshold.

- **Direct Call (Movie Service):** Avg 12ms.
- **Through Gateway:** Avg 38ms
- **Conclusion :** The ~26ms overhead is acceptable given the security and ratelimiting benefits provided at the edge.

1.2.2 Availability & Fault Tolerance (Circuit Breaker)

To test the **Resilience4j Circuit Breaker**, we simulated a failure in the customerservice by stopping its container.

- **Observation:** The Gateway detected the failure immediately upon reaching the error threshold.
- **Result:** Instead of a system-wide timeout, the Gateway returned a **Fallback Response** ("Service is temporarily unavailable") in under 200ms

1.2.3 Scalability (Docker-Compose Scaling)

Horizontal scaling was tested by increasing movie-service instances:

Bash: `docker-compose up -d --scale movie-service=3`

- **Evidence:** The Eureka Discovery Server correctly identified three active instances. The Gateway's Load Balancer automatically distributed traffic across all three.

1.3 End-to-End Testing (Flutter ↔ Gateway ↔ Microservices)

This validates the entire integration chain from the mobile client to the database

The "Purchase to Play" Flow Evidence:

1. **User Trigger:** User clicks "Upgrade to Premium" in the Flutter App
2. **Gateway Traffic:** Request passes through *AuthenticationFilter* and routes to *payment-service*
3. **Third-Party Integration:** Stripe Payment Sheet completes the transaction
4. **Async EDA:** *payment-service* publishes a message to **RabbitMQ**.
5. **Service Sync:** *notification-service* logs: *Consuming event: SubscriptionActivated*
6. **Final Verification:** Flutter App refreshes state; "Locked" movies become accessible.

2. Architectural Assessment & Lessons Learned

2.1 Fulfillment of FRs / NFRs / ASRs

The Nozie platform successfully fulfilled the requirements established during the planning phase:

- **Functional Requirements (FRs):** All core user stories—including secure authentication, content discovery, premium subscription management via Stripe, and asynchronous notifications—are fully operational across the FlutterMicroservices stack.
- **Non-Functional Requirements (NFRs):**
 - Scalability:** Achieved through the **Database per Service** pattern and **Docker-based horizontal scaling**, allowing the movie-service to scale independently based on demand.
 - Availability:** Enhanced via **Resilience4j Circuit Breakers** at the API Gateway, ensuring that a failure in the notification-service does not block the payment-service.
 - Security:** Centralized at the **API Gateway** with JWT validation and rate limiting, successfully protecting internal services from unauthorized or high-volume traffic.
- **Architecturally Significant Requirements (ASRs):** The decision to implement **Event-Driven Architecture (EDA) with RabbitMQ** was pivotal in ensuring eventual consistency between payments and notifications without sacrificing system responsiveness.

2.2 Monolith vs. Microservices: Trade-off Analysis

The transition from a monolithic approach to Microservices introduced significant benefits alongside specific operational challenges:

Attribute	Monolithic Approach (Not Used)	Nozie Microservices Architecture
Development	Simpler initial setup and debugging.	Higher complexity; requires service discovery and config servers.
Deployment	"All or nothing" deployment.	Independent deployment; reduces downtime risk.
Scalability	Scaled as a single unit (resource inefficient).	Fine-grained scaling (Movie Service scales independently).
Fault Tolerance	One bug can crash the entire application.	Fault Isolation: Payment failure does not affect Movie viewing.
Technology	Locked into a single tech stack.	Heterogeneous: Mix of MongoDB and Postgres based on need.
Operation	Easy to monitor (single log).	Harder to trace (requires Distributed Tracing).

2.3 Advantages, Limitations, and Remaining Risks

Advantages :

- **Robustness:** RabbitMQ ensures notifications are never lost, even if a service is temporarily offline
- **Clean Design:** The Flutter client's **Feature-First** architecture mirrors the backend, simplifying bug isolation.
- **Secured Edge: The API Gateway** provides a powerful "defense in depth" strategy

Limitations :

- **Operational Overhead:** Running 6+ microservices and infrastructure (Redis, RabbitMQ, DBs) requires significant local hardware resources
- **Network Latency:** Internal service-to-service calls (**via OpenFeign**) introduce slight latency compared to in-memory calls.

Remaining Risks :

- **Single Point of Failure:** The API Gateway currently runs as a single instance. Production requires a High Availability (HA) cluster.
- **Distributed Transactions:** The system relies on eventual consistency. Implementing "Saga Patterns" for multi-service transactions is a future challenge.

2.4 Lessons Learned

Key technical insights gained throughout the project:

- **Service Discovery** is Essential: In dynamic Docker environments, fixed IPs are impossible; Eureka proved vital for seamless communication.
- **Async over Sync:** Asynchronous events (RabbitMQ) are far more robust than synchronous REST calls for non-critical path tasks like notifications
- **Observability is Mandatory:** Implementing a Correlation ID across the Gateway and services was essential for tracing requests across the distributed system.
- **Flutter & Riverpod Efficiency:** Riverpod significantly simplified managing complex authentication states compared to native Flutter state management.

3. Conclusion & Future Direction

3.1 General Conclusion for Nozie Project

The Nozie Movie Platform project successfully demonstrated the implementation of a modern, enterprise-grade software architecture. By evolving into a Microservicesbased ecosystem, the platform has achieved a high degree of modularity, scalability, and resilience.

Key achievements include:

- **Infrastructure Excellence:** Successful integration of **Spring Cloud** components (Gateway, Eureka, Config Server) to efficiently handle service discovery and centralized security.
- **Data Integrity:** Implementation of the "**Database per Service**" pattern, ensuring each domain (Movie, Identity, Payment) maintains data sovereignty and prevents tight coupling.
- **Event-Driven Robustness:** Utilization of **RabbitMQ** for asynchronous communication, ensuring high reliability and decoupling between critical and non-critical system components.
- **Unified Client Experience:** A **Flutter** mobile application providing a premium, high-performance UI that interacts seamlessly with backend services through a single secure Gateway
- Overall, the project met all design objectives, proving that a microservices architecture provides a superior foundation for modern, high-traffic media platform

3.2 Functional and Architectural Extension Roadmap

The Nozie platform has a clear path for expansion to become a fully-featured market competitor:

3.2.1 Functional Extensions (New Features)

- **AI-Powered Personalization:** Developing a dedicated **ML-Recommendation Service** (using Python/FastAPI) for hyper-personalized, "Netflix-style" suggestions.
- **Social Engagement:** Adding a **Community Service** to handle movie reviews, user ratings, and social watch parties.
- **Multi-Format Support:** Expanding the **Movie Service** to support highdefinition **HLS adaptive bitrate streaming** for optimized playback.
- **Offline Viewing:** Implementing a download-to-local feature in the Flutter app using secure **DRM encryption**.

3.2.2 Architectural & Infrastructure Extensions

- **Kubernetes Orchestration (K8s):** Moving from Docker-Compose to Kubernetes for large-scale deployments with auto-healing and rolling updates.
- **Full Observability Stack:** Expanding current tracing into a full **ELK Stack** (Elasticsearch, Logstash, Kibana) and **Prometheus/Grafana** for real-time monitoring

- **Edge Caching:** Implementing CDN integration and advanced Redis caching at the Gateway level to improve global response times.
- **Saga Pattern Implementation:** Managing complex, distributed multi-service transactions (e.g., subscription bundling) to maintain data consistency

Appendices

Appendix A: Summary of Labs Content (1–10) The Nozie platform was developed incrementally through a series of structured labs, each focusing on a specific architectural milestone:

Lab / Week	Focus Area	Key Achievements
Lab 1 & 2	Monolithic Foundation	Defined basic entities; implemented CRUD for Movie Service using Spring Boot.
Lab 3 & 4	Microservices Split	Successfully decoupled the system into independent services; integrated Eureka for discovery.
Lab 5	API Gateway & Security	Implemented Spring Cloud Gateway ; centralized JWT Authentication and authorization.
Lab 6	Dockerization	Containerized all services and infrastructure (DBs, Redis, RabbitMQ) into a Docker Compose environment.
Lab 7	Event-Driven (EDA)	Integrated RabbitMQ ; established asynchronous communication between Payment and Notification.
Lab 8	Resilience & Performance	Configured Resilience4j circuit breakers and optimized routing throughput at the Gateway.
W9 & 10	Frontend & Integration	Finalized the Flutter mobile client; completed E2E integration testing and documentation.

Appendix B: Task Assignment & Progress Table Note: Please update the "Assignee" column with specific group member names

Module / Feature	Task Description	Assignee	Status
Backend Core	Movie, Identity, & Customer Services implementation	[Member 1]	100%
Infrastructure	API Gateway, Eureka, Config Server, & Docker Compose	[Member 2]	100%
EDA & Payment	RabbitMQ integration & Stripe Payment flow	[Member 3]	100%
Mobile Client	Flutter UI/UX, State Management, & API integration	[Member 4]	100%
Documentation	Report writing, Diagrams, & Architectural analysis	All Members	100%

C.1 API Gateway Route Configuration (application.yml)

YAML

```
spring:
  cloud:
    gateway:
      routes:
        - id: movie-service
          uri: lb://movie-service
          predicates:
            - Path=/api/movies/**
          filters:
            - name: CircuitBreaker
              args:
                name: movieService
                fallbackUri: forward:/fallback/movie
```

C.2 Event Publishing Logic (PaymentService.java)

Java

```
public void processPayment(PaymentRequest request) {
    // Process payment with Stripe ...

    // Publish success event to RabbitMQ
    rabbitTemplate.convertAndSend(
        "payment.exchange",
        "subscription.activated",
        new SubscriptionEvent(request.getUserId(), "PREMIUM")
    );
}
```

C.2 Event Publishing Logic (PaymentService.java)

```
Java
public void processPayment(PaymentRequest request) {
    // Process payment with Stripe ...

    // Publish success event to RabbitMQ
    rabbitTemplate.convertAndSend(
        "payment.exchange",
        "subscription.activated",
        new SubscriptionEvent(request.getUserId(), "PREMIUM")
    );
}
```

Appendix D: System Run Guide & Repo Links

D.1 Running the System Locally

To launch the entire Nozie ecosystem, follow these steps:

1. **Prerequisites:** Ensure **Docker Desktop** and **Java 17+** are installed.
2. **Environment Setup:** Navigate to the root directory of the server.
3. **Command Execution:**

Bash

cd Src/server/microservices
docker-compose up -d --build
4. **Verification:**
 - o **Eureka Dashboard:** <http://localhost:8761>
 - o **API Gateway:** <http://localhost:8080>
5. **Run Client:** Open the Src/client folder in VS Code/Android Studio and run flutter run.

D.2 Repository Links

- **Main Project Repo:** [Insert GitHub/GitLab Link Here]
- **Documentation Folder:** /Documents/
- **API Collection:** /Documents/Postman/Nozie_API_Collection.json